Distance Analysis Framework

> This repository contains a MATLAB-based framework for solving inverse static problems in compliant mechanisms (such as finding the force required to achieve a specific displacement). The process essentially uses an optimization loop to find out exactly how much moment (torque) is required to push the mechanism to a specific target angle.

Table of Contents

1. System Architecture: The Three Layers
2. The Core Algorithm (Bi-level Optimization)
3. Solving the Ex6 Problem (Example Workflow)
4. Proposed Architectural Improvements (Path to FEA Accuracy)
5. Inner Solver Operations & Newton-Raphson Integration

1. System Architecture: The Three Layers

The framework operates on a three-layered system to solve inverse static problems:

- **Kinematics (The Skeleton & Rules of Motion):** Acts as the foundational geometry and constraint manager. It builds the network (nodes/links), finds constraints (loops) using graph theory, and calculates valid geometric configurations.
- **Statics (The Muscle & Equilibrium):** Builds on top of `Kinematics`. It adds stiffness (using Pseudo-Rigid-Body or CBCM models) and performs forward simulations. It calculates where the mechanism will settle under a specific known load.
- **DistanceAnalysis (The Brain & Reverse-Engineer):** Sits at the top. It acts as an optimization wrapper that uses `Statics` and `Kinematics` to reverse-engineer the required force or moment to reach a target geometric configuration.

Deep Dive: Architectural Components

1. Kinematics: The Skeleton and Rules of Motion

The Kinematics class acts as the foundational geometry and constraint manager. It doesn't care about

forces, materials, or springs; it only cares about how things are allowed to move. When the mechanism is built, Kinematics does the following:

- **Builds the Network:** It uses methods like `constructLinksNodes` and `getAdjacencyMatrix` to figure out exactly how every node and link is connected.
- **Finds Constraints (Loops):** Through `findLoops()` and `kinematicEquations()`, it defines the mathematical rules that say, "If Node 2 moves here, Node 3 must move there because they are connected by a rigid pin."
- **Calculates Configurations:** If you try to move a part of the mechanism, Kinematics uses an internal solver (`fsolve` inside `findNewConfig`) to calculate the exact new (X, Y, Theta) coordinates of every node while strictly obeying the joint constraints.

2. Statics: The Muscle and Equilibrium

The Statics class (which works closely with LoadAnalysis) builds on top of Kinematics. While Kinematics knows how it can move, Statics knows how hard it is to move.

- **Adds Stiffness:** It looks at the compliance you added (like the `BeamType.PRB` pseudo-rigid-body models in Ex6). It knows these act like springs.
- **Forward Simulation:** If you explicitly apply a 50 N force to the mechanism, the Statics class takes over. It pushes the mechanism, asks Kinematics for the valid geometric paths, and calculates where the mechanism will settle once the external force (50 N) perfectly balances the internal spring resistance.
- **In short:** Statics answers the question: *"If I apply a specific known load, what is the final deformed shape?"*

3. DistanceAnalysis: The Brain and Reverse-Engineer

The DistanceAnalysis class sits at the very top. In real-world engineering (like your Ex6 script), you rarely know the exact force you need; instead, you know where you want the mechanism to go and need to find the force required to get there. DistanceAnalysis acts as an optimization wrapper that uses Statics and Kinematics to reverse-engineer the answer.

Here is the exact loop it runs during `simulationNoGUI`:

- **The Target:** You tell DistanceAnalysis your goal (e.g., "I want Link 1 to move exactly +5mm in the X direction").
- **The Guess:** The optimizer inside DistanceAnalysis makes a wild guess: *"Let's try applying 10 Newtons."*
- **The Delegation (Statics + Kinematics):** It passes that 10 N guess down to the Statics class. Statics (relying on Kinematics for geometry) runs a forward simulation and reports back: *"Under 10 N of force, Link 1 only moved +3mm."*

- **The "Distance":** DistanceAnalysis compares the result (+3mm) to the target (+5mm). The error (or "distance") is 2mm.
- **Iteration:** Because the distance is not zero, the optimizer adjusts its guess: *"10 N wasn't enough. Let's try 18 Newtons."* It loops this process, continuously running Statics simulations, until the distance is practically zero.

Summary: Kinematics provides the geometric boundaries. Statics provides the physical resistance and forward calculation. DistanceAnalysis treats them both as a "black box," repeatedly guessing forces until the output perfectly matches your target input.

2. The Core Algorithm (Bi-level Optimization)

Currently, the framework operates on a **Bi-level Optimization Algorithm**:

1. **The Outer Loop (Nelder-Mead Simplex):** Handled by `DistanceAnalysis` using MATLAB's `fminsearch`. It guesses a force to minimize the error between the simulated position and the target position.
2. **The Inner Loop (Minimum Potential Energy):** Handled by `LoadAnalysis` using `fmincon`. For every force guessed by the outer loop, it calculates the physical static equilibrium by minimizing the system's total potential energy (internal strain + external work).
3. **The Foundation (Loop Closure):** Enforced by `Kinematics`, it uses loop closure equations as non-linear constraints to prevent the inner optimizer from breaking joints or stretching rigid links.

Deep Dive: Algorithm Mechanics

Based on the code provided in the framework (specifically looking at `DistanceAnalysis.m`, the Kinematics setup, and the calls to `LoadAnalysis`), the entire framework operates on a Bi-level Optimization Algorithm (or nested optimization). Instead of solving a single massive equation, the framework stacks solvers on top of each other. Here is the mathematical and algorithmic engine driving the whole thing, broken down from the top level to the foundation:

1. The Outer Loop: Nelder-Mead Simplex Algorithm (Inverse Statics)

- **Where it lives:** `DistanceAnalysis.m` (specifically using MATLAB's `fminsearch`).
- **What it does:** Solves the "Reverse-Engineering" problem.
  When you say, "I want the mechanism to move exactly 5mm, what force do I need?", DistanceAnalysis defines an objective function. This function calculates the Sum of Squared Errors between the target position and the actual simulated position. To minimize this error down

to zero, it uses `fminsearch`, which is MATLAB's implementation of the Nelder-Mead Simplex algorithm. This is a heuristic, derivative-free optimization method. It basically creates a "triangle" of guessed forces, evaluates the error for each, and mathematically "flips" the triangle down the slope of the error curve until it reaches the bottom (zero error).

2. The Inner Loop: Principle of Minimum Potential Energy (Forward Statics)

- **Where it lives:** `LoadAnalysis.m` (called via `loadSimulation.fMinConMain` inside the distance analysis loop).
- **What it does:** Solves the "Equilibrium" problem.
  Every time the Nelder-Mead algorithm guesses a force (e.g., 10 Newtons), it has to figure out how the mechanism reacts. To do this, it hands the 10N force down to LoadAnalysis. The algorithm used here is based on the Principle of Minimum Potential Energy. The system calculates:
- Internal Strain Energy: The energy stored in the compliant links (the PRB models acting as torsion springs).
- External Work: The work done by the applied forces/moments.
  It uses MATLAB's `fmincon` (Constrained Non-linear Minimization, typically using Interior-Point or Sequential Quadratic Programming (SQP) algorithms) to find the exact physical shape where the total energy of the system is at its absolute minimum. That minimum energy state is the physical static equilibrium.

3. The Foundation: Graph Theory and Loop Closure (Kinematics)

- **Where it lives:** `Kinematics.m`.
- **What it does:** Provides the strict mathematical boundaries that `fmincon` must obey.
  Before any energy can be minimized, the framework uses Graph Theory (specifically an adjacencyMatrix and findLoops) to map out the mechanism. It converts the physical connections into Kinematic Loop Closure Equations. These equations state that if you trace a path from the ground, through the links, and back to the ground, the sum of the vectors must equal zero.
  When the inner loop (`fmincon`) is trying to minimize the energy of the springs, the Kinematics class enforces these loop closure equations as non-linear constraints. It physically prevents the optimizer from guessing a state where a pin joint is "broken" or a rigid link is stretched.

Summary of the Flow:

- Outer Optimizer (`fminsearch`) guesses a Force.
- Inner Optimizer (`fmincon`) tries to minimize the system's potential energy under that Force...
- ...while strictly obeying the Loop Closure constraints generated by Graph Theory.
- Once equilibrium is found, the Outer Optimizer checks if the new position matches your target. If not, it updates the guess and repeats the cycle.

3. Solving the Ex6 Problem (Example Workflow)

Distance Analysis: Solving the Ex6 Problem

Here is a step-by-step breakdown of how the classes solve a specific moment-angle problem (Ex6):

1. **Building the Physical Model:** `Node` and `Link` classes define the rigid body geometry. `makeCompliant` modifies specific links to resist motion like a spring.
2. **Setting up the Distance Analysis:** `addInput` defines the kinematic target (e.g., target angle). `selectLoad` defines the unknown variable (e.g., the moment magnitude).
3. **The Core Solving Engine:** `simulationNoGUI` runs the bi-level optimization. It iteratively guesses the moment, runs forward statics, measures the distance to the target angle, and repeats until the error is practically zero.
4. **Extracting the Results:** The final computed load is extracted and plotted against the target displacement to generate a Force-Displacement (or Moment-Angle) curve.

Here is the step-by-step breakdown of how the classes achieve this:

3.1 Building the Physical Model (Design Classes)

Before any solving happens, the script uses the basic building block classes to construct the physical mechanism:

- **Node & Link**: These define the rigid body geometry of the Four-Bar Mechanism.
- **Compliant Modification**: Link 3 is modified using `makeCompliant(..., BeamType.PRB)`. This means the system is no longer purely rigid; it will resist motion like a spring, requiring force/moment to move.
- **Moment**: A baseline moment object is attached to Link 1. Initially, its magnitude is set to `1`, but this is just a placeholder that the solver will manipulate later.

3.2 Setting up the Distance Analysis

You pass these design elements into the `DistanceAnalysis` class. This class acts as the "manager" for inverse static problems (finding required loads for desired positions). Inside your 10-step loop, you configure two critical things using `DistanceAnalysis` methods:

- **`addInput(..., 'angle', ...)`** : This defines your **Kinematic Target**. You are telling the system, "I want Link 1 to reach this specific angle."
- **`selectLoad(..., 'moment')`** : This defines your **Unknown Variable**. You are telling the system, "You are allowed to change the magnitude of the Moment on Link 1 to achieve the target angle."

3.3 The Core Solving Engine (`simulationNoGUI`)

When you call `analysis.simulationNoGUI(currentWorkspace)` , the real math happens. Based on the methods visible in your class diagram (`findDistanceAnalysis` , `outputFunction` , `state` , `optimValues`), `DistanceAnalysis` solves the problem using an optimization algorithm (likely MATLAB's `fminsearch` or a similar optimizer).

Here is what happens under the hood during that method call:

- **The Guess**: The optimizer guesses a magnitude for the unknown Moment.
- **Forward Simulation**: `DistanceAnalysis` hands this guessed moment off to the `LoadAnalysis` and `Statics` classes. These classes calculate the forward statics—meaning they figure out how the mechanism deforms and settles into an equilibrium state under that specific guessed load.
- **Measuring the "Distance"**: Once equilibrium is found, the system checks the new angle of Link 1 and compares it to your target angle (the one set by `addInput`). The difference between the actual angle and the target angle is the "Distance" (or error).
- **Iteration**: The optimizer uses `findDistanceAnalysis` to evaluate this error. It continuously adjusts the guessed Moment, running the statics simulation over and over, until the error is practically zero (i.e., the simulated angle perfectly matches your target angle).

3.4 Extracting the Results

Once the optimizer converges, the final correct moment magnitude is saved into the `analysis.newValues.newValue` property. Your script then extracts this computed load (`forces(i) = analysis.newValues.newValue`) and the corresponding target displacement, storing them to plot the final Force-Displacement (or in this case, Moment-Angle) curve.

In short, `DistanceAnalysis` acts as a reverse-engineering wrapper around `LoadAnalysis` . Instead of asking "What happens if I apply 50Nm?", it uses an optimizer to ask "What load do I need to apply to make the mechanism rotate exactly 20 degrees?"

4. Proposed Architectural Improvements (Path to FEA Accuracy)

To improve the speed, accuracy, and numerical stability of this framework to rival commercial FEA software, the following architectural refactors are recommended:

- **Eliminate the Nested Outer Loop (Displacement Control):** Instead of using `fminsearch` to guess forces, apply a *prescribed displacement* boundary condition directly to the target node. Solve the equilibrium state once, and the required force is simply the reaction force at that displaced node.
- **Ditch `fmincon` for Newton-Raphson:** Minimizing potential energy is slow. Implement a standard Newton-Raphson solver to find the roots of the equilibrium equations directly ($\mathbf{R}(\mathbf{u}) = \mathbf{0}$) using the Tangent Stiffness Matrix ($\mathbf{K}_T$).
- **Upgrade the Element Formulation:** Transition from Pseudo-Rigid-Body (PRB) models to continuous Nonlinear FEA Beam Elements (like Co-Rotational or Geometrically Exact Reissner-Simo beams) to accurately capture large deflections and combined stress states.
- **Better Handling of Follower Forces:** A Newton-Raphson solver naturally accommodates the Load Stiffness Matrix, enabling the solution of follower forces in a single increment without artificially forcing tiny step increments.

Deep Dive: Refactoring Guide

Currently, your framework relies heavily on Nested Optimization (using `fminsearch` wrapped around `fmincon`). While this is logically intuitive (guess a force $\rightarrow$ find equilibrium $\rightarrow$ check distance $\rightarrow$ guess again), it is computationally disastrous and numerically unstable. Commercial FEA software almost never uses energy minimization (`fmincon`) as the primary solver, and they certainly never nest optimizers.

Here is a deep dive into what should be rewritten and the architectural changes needed to make this framework faster and more accurate.

1. Eliminate the Nested Outer Loop (The Speed Fix)

- **The Problem:** DistanceAnalysis uses `fminsearch` (Nelder-Mead algorithm) to guess the force. For every single guess, it calls LoadAnalysis, which uses `fmincon` to find the minimum potential energy. If `fminsearch` takes 50 iterations, and `fmincon` takes 50 iterations per guess, you are doing 2,500 full system evaluations for one step.
- **The Solution:** Implement Displacement Control In standard FEA, if you want to know the force required to move a node by 5mm, you do not guess the force. Instead, you use Displacement

Control.

  - You apply a prescribed displacement boundary condition directly to the node (just like a ground pin, but moved 5mm).
  - You solve the equilibrium state once.
  - The required force is simply calculated as the Reaction Force at that displaced node.
- **How to implement it:** Instead of DistanceAnalysis acting as an outer loop, it should modify the Kinematics constraints.
  - Add the target displacement as a strict equality constraint in your inner solver.
  - Run the solver exactly once.
  - Extract the Lagrange multiplier (or use the tangent stiffness matrix) associated with that constraint—that multiplier is your required force/moment.

2. Ditch fmincon for Newton-Raphson (The Standard FEA Architecture)

- **The Problem:** Minimizing potential energy using `fmincon` (which likely uses SQP or Interior-Point methods) is very slow for structural mechanics.
- **The Solution:** Root-finding via the Tangent Stiffness Matrix. Instead of minimizing energy, you should solve the equilibrium equations directly: $\mathbf{R}(\mathbf{u}) = \mathbf{0}$, where $\mathbf{R}$ is the residual force vector (Internal Forces - External Forces).
  - Assembly: Calculate the internal force vector and the Tangent Stiffness Matrix ($\mathbf{K}_T$) for all elements.
  - Newton-Raphson Iteration: Update the displacements using $\Delta\mathbf{u} = \mathbf{K}_T^{-1}\mathbf{R}$.
  - Convergence: Loop this 3 to 5 times until $\mathbf{R} \approx 0$.
    This is how ANSYS, Abaqus, and SolidWorks operate. A custom Newton-Raphson solver in MATLAB will run 10x to 100x faster than `fmincon` and won't require you to tune optimization tolerances.

3. Upgrade the Element Formulation (The Accuracy Fix)

- **The Problem:** The current script heavily relies on the `BeamType.PRB` (Pseudo-Rigid-Body) model. The PRB model is an approximation that assumes a flexible beam can be modeled as two rigid links connected by a torsion spring. While great for quick kinematics, it is mathematically incapable of matching true FEA accuracy for large, continuous deflections, complex stress states, or combined axial/bending loads.
- **The Solution:** Nonlinear FEA Beam Elements To get FEA-level accuracy, you must implement true continuous mechanics.
  - Co-Rotational Beam Elements: This formulation separates rigid body motion from local elastic deformation. It allows you to use simple linear beam theory (Euler-Bernoulli) at the

local level, but mathematically rotates the element to handle massive geometric nonlinearities (large deflections).
    - Geometrically Exact Beams (Reissner-Simo): If you need the ultimate accuracy for highly compliant mechanisms, this formulation handles shear deformation, axial stretching, and extreme bending perfectly.
- **Note:** Your code mentions `BeamType.CBCM` (Chained Beam Constraint Model). If fully implemented, CBCM is much closer to FEA accuracy than PRB. Ensure your solver is defaulting to this for accuracy benchmarks.

4. Better Handling of Follower Forces

- **The Problem:** In `findDistanceAnalysis`, there is a conditional block that forces `increments = 10` if a follower force is detected, and `1` otherwise. Follower forces (forces that change direction as the mechanism deforms, like pressure) make the stiffness matrix asymmetric.
- **The Solution:** By moving to a standard Newton-Raphson solver, you can naturally include the Load Stiffness Matrix (the derivative of the force vector with respect to displacement). This allows you to solve follower forces in a single increment without manually forcing the system to take 10 tiny steps.

5. Inner Solver Operations & Newton-Raphson Integration

Forward Statics Deep Dive

While DistanceAnalysis guesses what force to apply, solveStaticAnalysis answers the question: "Given this specific force and these compliant joints, what is the final, deformed shape of the mechanism?" Here is a breakdown of its specific contributions and how it works mathematically:

1. The Core Algorithm: Constrained Energy Minimization
Instead of using standard force-balance equations ($\Sigma F = 0$), this script finds equilibrium using the Principle of Minimum Potential Energy. It uses MATLAB's fmincon (with the interior-point algorithm) to find the configuration of the mechanism that has the lowest possible energy state.
It does this by balancing two opposing factors (calculated inside fMinConFunc):

- Internal Strain Energy: The energy absorbed by the springs and compliant beams (PRB, CBCM, Mlinear) as they bend. The script wants to keep this low (meaning it resists bending).

- External Work: The energy added to the system by the external forces and moments pushing on the nodes.

2. Enforcing Physical Reality (The Constraints)

If the solver only minimized energy, the mechanism would just tear itself apart to relieve the stress on the springs. To prevent this, solveStaticAnalysis uses the fMinConCostraint function.

- This function enforces the Kinematic Loop Closure Equations.
- It tells fmincon: "You can bend the links to lower the energy, but the X and Y distances between these pinned connections must always equal exactly zero."
- This ensures the mechanism stays physically assembled while it deforms.

3. High-Performance Analytical Derivatives

The biggest technical achievement inside this file is the immense amount of calculus hardcoded into it. Optimization algorithms like fmincon usually run very slowly if they have to guess the slope of the equations (Finite Differences). To speed this up, solveStaticAnalysis provides exact, analytical derivatives:

- Gradients: Functions like prbGradient, bcmGradient, and linearGradient provide the exact first derivatives (Jacobians) of the energy equations.
- The Hessian: The hessianfcn builds the exact second-derivative matrix (the Tangent Stiffness Matrix) of the entire system.
  By feeding the exact Gradients and Hessian directly into fmincon, the solver can aggressively jump to the minimum energy state rather than feeling its way around blindly.

NewtonRaphsonSolver Integration (The KKT Matrix)

The `NewtonRaphsonSolver` completely changes the mathematical approach. It upgrades the framework from a slow "guess-and-check" optimization strategy to a direct, high-speed mathematical root-finding strategy, acting as a **Constrained Multi-Variable FEA Solver**.

Instead of minimizing energy, it finds the roots (zeros) of the force residual equations ($\mathbf{R}(\mathbf{x}) = \mathbf{F}_{internal}(\mathbf{x}) - \mathbf{F}_{external} = 0$). Here is how it solves the Ex6 problem using the Newton-Raphson equation:

1. The Components of the Newton-Raphson Step

At every iteration, the solver pulls four exact analytical components from `LoadAnalysis` and `Kinematics`：

- g **(The Gradient):** Internal spring forces (1st derivative of energy).
- H **(The Hessian):** Tangent Stiffness Matrix (2nd derivative of energy).

- ceq **(Kinematic Constraints):** Current error in joint connections.
- J **(The Jacobian):** 1st derivative of the kinematic constraints.

2. Assembling the KKT Matrix & Residual

The solver assembles a massive KKT (Karush-Kuhn-Tucker) Matrix and a Residual vector to determine how far off the current guess is from perfect equilibrium:

- **KKT Matrix:** $\begin{bmatrix} \mathbf{H} & \mathbf{J}^T \\ \mathbf{J} & \mathbf{0} \end{bmatrix}$ (Top-left handles elastic stiffness, off-diagonals enforce rigid physical rules).
- **Residual:** $\begin{bmatrix} \mathbf{g} + \mathbf{J}^T \lambda \\ \mathbf{ceq} \end{bmatrix}$ (Top half checks force balance, bottom half checks if joints are sealed).

The solver then executes the core Newton-Raphson update step using MATLAB's matrix left-division (`\`):

$$\begin{bmatrix} \Delta \mathbf{x} \\ \Delta \lambda \end{bmatrix} = -\mathbf{KKT} \setminus \mathbf{Residual}$$

3. The Magic of Displacement Control (Zero Guessing)

The most brilliant part of this solver is how it solves the **Inverse Statics Problem** (e.g., finding the force needed for a 5mm displacement) *without any outer guessing loop*.

When a target displacement is applied, the solver adds one extra row to the $\mathbf{J}$ matrix and one extra constraint to the ceq vector representing that 5mm target. The external force parameter is explicitly set to `0`. When the Newton-Raphson loop finishes converging ($\mathbf{Residual} \approx 0$), **the required external force is simply plucked out of the λ vector** as the Lagrange multiplier associated with that specific displacement constraint. This completely eliminates the outer `fminsearch` guessing loop, dropping calculation times drastically!